



FluxoSDK

Guia Explicativo do Código

Estudo técnico do monorepo, cobrindo arquitetura, termos, blocos de código, banco de dados, SDK de captura, backend, dashboard Angular, cliente de exemplo, fluxos principais e pontos de atenção.

Data

05/05/2026

Workspace

C:\Users\luizr\work\FluxoSDK_Refatorado

Stack principal

Nx, Angular standalone, Express, PostgreSQL, Chart.js, Jest, Playwright

Escopo analisado

Código e configuração de `apps/backend`, `apps/frontend`, `apps/example-client`, e2e e configs raiz. Dependências, builds e arquivos gerados foram tratados como suporte.

Sumário

1. Resumo executivo
2. Inventário do repositório
3. Glossário técnico
4. Arquitetura geral
5. Backend Express e PostgreSQL
6. Banco de dados
7. SDK embarcado `embed.js`
8. Frontend Angular
9. Cliente de exemplo
10. Fluxos de ponta a ponta
11. Testes e qualidade
12. Pontos de atenção e melhorias

1. Resumo Executivo

O FluxoSDK é uma plataforma experimental de analytics comportamental. Ele tem três partes principais: um SDK JavaScript que roda no site monitorado, uma API Express que recebe e agrega eventos, e um painel Angular que transforma esses eventos em métricas visuais.

Ideia central: o site cliente carrega `/assets/embed.js` com uma `siteKey`. Esse script registra eventos de navegação, clique, formulário, scroll, performance e erro. A API valida a chave pública, grava os eventos no PostgreSQL e o dashboard consulta estatísticas por site e período.

O que o sistema já entrega

- Cadastro e login de usuários.
- Cadastro de sites e geração de chaves públicas `pk_...`.
- Coleta de eventos via script externo.
- Armazenamento de sessões, visitas por página e eventos detalhados em JSONB.
- Dashboard com métricas de page views, visitantes, sessões, erros, formulários, cliques problemáticos, Web Vitals, fontes, dispositivos, funis e fluxos.
- Área administrativa para usuários root.
- Cliente de exemplo com rotas e ações para gerar dados de teste.

Mapa mental rápido

```
Site monitorado
  carrega <script src="http://localhost:3000/assets/embed.js" data-site-key="pk_...">
    |
    v
FluxoSDK no navegador
  gera visitorId, sessionId, pageId, eventId
  coleta page_view, click, form_*, web_vital, errors, custom
    |
    v
Backend Express
  PUT/POST /api/sdk/events
  valida siteKey, transaciona, grava eventos e índices de visita
    |
    v
PostgreSQL
  users, sites, site_keys, browser_sessions, page_visits, sdk_events
    |
    v
Frontend Angular
  /dashboard consulta /api/sdk/stats e /api/sdk/events/recent
  renderiza KPIs, gráficos, listas e área admin
```

2. Inventário do Repositório

O projeto é um monorepo Nx. A raiz contém configurações compartilhadas e cada app tem seu próprio `project.json`, `tsconfig`, ESLint e Jest/Playwright quando aplicável.

Área	Arquivos principais	Responsabilidade
Raiz Nx	<code>package.json</code> , <code>nx.json</code> , <code>tsconfig.base.json</code> , <code>eslint.config.mjs</code>	Scripts, dependências, targets, cache, lint, TypeScript base e Jest global.
Backend	<code>apps/backend/src/main.ts</code> , rotas, controllers, services, <code>database/schema.sql</code> , <code>assets/embed.js</code>	API Express, conexão PostgreSQL, ingestão de eventos, consultas analíticas e entrega do SDK.
Frontend	<code>apps/frontend/src/app/pages/*</code> , services, guards, routes	Landing, autenticação, dashboard, gráficos, métricas, sites monitorados e admin.
Example client	<code>apps/example-client/src/index.html</code> , <code>app.routes.ts</code>	Aplicação de laboratório que carrega o SDK e dispara eventos reais para testar métricas.
Testes	<code>apps/backend-e2e</code> , <code>apps/frontend-e2e</code> , specs Angular	Estrutura de teste criada pelo Nx; parte dela ainda aponta para comportamento inicial de scaffold.

Scripts disponíveis

```
{
  "dev": "nx run-many --target=serve -p frontend backend example-client --parallel=3",
  "back": "nx run backend:serve",
  "front": "nx run frontend:serve",
  "example": "nx run example-client:serve --port=4002"
}
```

O fluxo de desenvolvimento esperado é subir API, painel e cliente exemplo em paralelo. O frontend e o example-client chamam a API em `http://localhost:3000`.

3. Glossário Técnico

Nx

Ferramenta de monorepo. Organiza vários apps, cacheia tarefas e padroniza build, test, lint e serve.

Angular standalone

Componentes Angular sem módulos clássicos. Cada componente declara seus próprios imports.

Signal

Primitiva reativa do Angular. O dashboard usa `signal()` para estado local como aba ativa, sites e métricas.

Express

Framework HTTP usado no backend para rotas, controllers e middleware.

PostgreSQL / pg

Banco relacional e cliente Node usado por `pg.query()`.

JSONB

Tipo PostgreSQL para JSON indexável. O projeto guarda `metadata` e `context` de eventos nesse formato.

siteKey

Chave pública `pk_...` que liga um site monitorado ao backend.

visitorId / sessionId / pageId / eventId

IDs criados pelo SDK para distinguir visitante, sessão, página visitada e evento individual.

sendBeacon

API do navegador usada para tentar enviar dados quando a página está fechando.

PerformanceObserver

API usada pelo SDK para capturar Web Vitals e métricas de renderização.

Dead click

Clique em elemento aparentemente não acionável ou sem mudança detectável.

Rage click

Sequência de cliques repetidos no mesmo seletor em janela curta, sinalizando frustração.

4. Arquitetura Geral

A arquitetura segue uma divisão simples: SDK de captura no navegador, backend como API e agregador, PostgreSQL como fonte de verdade e dashboard como camada de visualização.

```
apps/backend
  main.ts
    cria app Express
    conecta no PostgreSQL
    roda schema.sql
    expõe /assets e /api

  routes/*
```

```
index.ts -> /sdk, /sites, /auth, /admin

controllers/*
  traduz HTTP para chamadas de serviço

services/*
  regras de negócio e SQL

assets/embed.js
  SDK carregado por sites externos

apps/frontend
  landing -> página inicial comercial/demo
  auth -> login/cadastro
  dashboard -> métricas, sites, admin e eventos

apps/example-client
  app real de teste com script embed e rotas que disparam eventos
```

Rotas HTTP do backend

Endpoint	Método	Descrição
/api/auth/register	POST	Cadastra usuário.
/api/auth/login	POST	Autentica usuário e retorna dados públicos.
/api/sites	GET	Lista sites do usuário ou todos se root.
/api/sites	POST	Cadastra site e gera <code>siteKey</code> .
/api/sites/:id	DELETE	Remove site se usuário tiver permissão.
/api/sdk/events	PUT/POST	Recebe lote de eventos do SDK.
/api/sdk/events/recent	GET	Lista eventos recentes.
/api/sdk/stats	GET	Retorna pacote completo de métricas do dashboard.
/api/admin/overview	GET	Painel root com usuários, sites e totais globais.

Bloco central de roteamento

```
const routes = Router();

routes.use('/sdk', sdkRoutes);
routes.use('/sites', sitesRoutes);
routes.use('/auth', authRoutes);
routes.use('/admin', adminRoutes);
```

5. Backend Express e PostgreSQL

Inicialização em `main.ts`

O backend cria um `Client` do PostgreSQL usando variáveis `BD_HOST`, `BD_PORT`, `BD_USER`, `BD_PASSWORD`, `BD_DATABASE` ou `BD_NAME`. Depois configura CORS aberto, JSON limitado a 256 KB, log de tráfego, arquivos estáticos em `/assets` e rotas em `/api`.

```
export const pg = new Client({
  host: process.env.BD_HOST,
  port: process.env.BD_PORT ? Number(process.env.BD_PORT) : undefined,
  user: process.env.BD_USER,
  password: process.env.BD_PASSWORD,
  database: process.env.BD_DATABASE ?? process.env.BD_NAME,
  connectionTimeoutMillis: process.env.BD_TIMEOUT_MS
    ? Number(process.env.BD_TIMEOUT_MS)
    : 5000,
});
```

Na subida, o servidor chama `connectPostgres()`, tenta conectar, roda `database/schema.sql` e então escuta a porta definida em `API_PORTA`.

Padrão `controller/service`

Os controllers validam entrada HTTP e retornam JSON. Os services concentram regra de negócio e SQL. Isso aparece em todas as áreas: Auth, Sites, SDK e Admin.

```
// Controller: recebe HTTP
receiveEvents = async (request: Request, response: Response) => {
  const result = await this.sdkService.receiveEvents(request.body);
  return response.status(202).json({ ok: true, data: result });
};

// Service: valida payload, acessa banco e decide o que gravar
async receiveEvents(input: ReceiveSdkEventsPayload) {
  // valida siteKey, sessionId e eventos
  // confere site_keys + sites
  // inicia transação
  // grava sessão, eventos e índices de visita
}
```

Autenticação atual

`AuthService` cadastra usuário com `crypt($2, gen_salt('bf'))`, função do `pgcrypto`, e faz login comparando `password_hash = crypt($2, password_hash)`.

O frontend grava `fluxosdk_user_id`, nome, e-mail e flag root no `localStorage`. As APIs protegidas recebem o usuário via header `x-user-id`.

Atenção: isso funciona como autenticação simplificada para ambiente local, mas não é segurança de produção. Qualquer cliente poderia enviar outro `x-user-id`. O caminho natural é JWT ou sessão assinada.

Cadastro de sites

`SitesService.registerSite()` limpa o domínio removendo protocolo e path, grava `sites` e gera uma chave pública com `crypto.randomUUID()`.

```
const cleanDomain = domain.replace(/^https?:\/\//, '').split('/')[0];
const publicKey = `pk_${crypto.randomUUID().replace(/-/g, '')}`;
```

Usuários root veem todos os sites. Usuários comuns só veem sites cujo `sites.user_id` é o seu próprio ID.

6. Banco de Dados

O arquivo `apps/backend/src/database/schema.sql` é executado automaticamente na subida. Ele cria a extensão `pgcrypto`, tabelas principais, migrações aditivas por `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` e índices de consulta.

Tabela	Papel	Campos importantes
<code>users</code>	Contas de acesso ao painel.	<code>email</code> , <code>password_hash</code> , <code>is_root</code> , <code>is_active</code> , <code>last_login_at</code>
<code>sites</code>	Propriedades monitoradas.	<code>user_id</code> , <code>name</code> , <code>domain</code> , <code>active</code>
<code>site_keys</code>	Chaves públicas de integração.	<code>site_id</code> , <code>public_key</code> , <code>active</code> , <code>revoked_at</code>
<code>browser_sessions</code>	Sessões vistas pelo SDK.	<code>site_key</code> , <code>visitor_id</code> , <code>session_id</code> , <code>context</code> , <code>last_seen_at</code>
<code>page_visits</code>	Índice de entrada/saída por página.	<code>page_id</code> , <code>entered_at</code> , <code>left_at</code> , <code>duration_ms</code> , <code>max_scroll_depth</code>
<code>sdk_events</code>	Log canônico dos eventos.	<code>event_id</code> , <code>event_type</code> , <code>event_name</code> , <code>metadata</code> , <code>context</code> , <code>occurred_at</code>

Trecho simplificado do modelo

```
CREATE TABLE IF NOT EXISTS sdk_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id TEXT NOT NULL UNIQUE,
  site_key TEXT NOT NULL,
  visitor_id TEXT NULL,
  session_id TEXT NOT NULL,
  page_id TEXT NOT NULL,
  event_type TEXT NOT NULL,
  event_name TEXT NULL,
  url TEXT NOT NULL,
  path TEXT NOT NULL,
  title TEXT NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL,
  metadata JSONB NOT NULL DEFAULT '{}':::jsonb,
  context JSONB NOT NULL DEFAULT '{}':::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```


Índices

Há índices para `site_key`, `session_id`, `visitor_id`, `page_id`, `event_type`, `occurred_at` e o índice composto `(site_key, event_type, occurred_at DESC)`. Isso sustenta as consultas do dashboard por site, tipo e período.

7. SDK Embarcado `embed.js`

`embed.js` é o script entregue em `/assets/embed.js`. Ele é autoexecutável: ao carregar, lê atributos do próprio `<script>`, cria IDs persistentes e começa a coletar eventos.

Como integrar em um site

```
<script
  src="http://localhost:3000/assets/embed.js"
  data-site-key="pk_ea22c699be734efbb305a4f9316f1cb9"
  data-track-api-errors="true">
</script>
```

Configurações lidas pelo SDK

Atributo	Uso
<code>data-site-key</code>	Obrigatório. Identifica o site no backend.
<code>data-api-url</code>	Opcional. Define base da API; senão usa a origem do script.
<code>data-sample-rate</code>	Opcional. Amostragem de 0 a 1.
<code>data-flush-ms</code>	Intervalo de envio, limitado entre 1000 e 30000 ms.
<code>data-batch-size</code>	Tamanho do lote, limitado entre 5 e 100.
<code>data-debug</code>	Ativa logs de debug.
<code>data-track-api-errors</code>	Intercepta <code>fetch</code> para registrar falhas de API.

Eventos capturados

- page_view
- route_change
- page_leave
- click
- dead_click
- rage_click
- form_start
- form_submit
- form_abandon
- field_error
- scroll_depth
- performance
- web_vital
- js_error
- resource_error
- api_error
- custom
- identify

Formato do payload enviado

```
{
  "siteKey": "pk_...",
  "visitorId": "vis_...",
  "sessionId": "sess_...",
  "userIdentifier": "user_12345",
  "sentAt": "2026-05-05T13:44:00.000Z",
  "context": {
    "sdkVersion": "0.2.0",
    "deviceType": "desktop",
    "browser": "Chrome",
    "os": "Windows",
    "utm": { "source": "google" }
  },
  "events": [
```

```

    {
      "eventId": "evt_...",
      "type": "click",
      "pageId": "page_...",
      "url": "http://localhost:4002/checkout",
      "path": "/checkout",
      "title": "example-client",
      "occurredAt": "2026-05-05T13:44:00.000Z",
      "metadata": { "selector": "button#start", "text": "Iniciar checkout" }
    }
  ]
}

```

Fila e envio

O SDK empilha eventos em `eventQueue`. Ele envia quando atinge `batchSize`, quando o timer de `flushMs` dispara, ou quando uma ação pede flush imediato. Em saída de página, tenta `navigator.sendBeacon` e cai para `fetch(..., keepalive)`.

```

function queueEvent(event, options) {
  eventQueue.push(event);

  if (eventQueue.length >= batchSize) {
    flush(false);
    return;
  }

  if (!flushTimer) {
    flushTimer = setTimeout(function () {
      flush(false);
    }, flushMs);
  }

  if (options && options.flush) {
    flush(Boolean(options.beacon));
  }
}

```

API pública no navegador

```

window.FluxoSDK = {
  version: VERSION,
  siteKey,
  apiUrl,
  visitorId,
  sessionId,
  getStatus() { ... },
  identify(id, traits) { ... },
  track(name, metadata) { ... },
  trackError(error, metadata) { ... },
  trackApiError(details) { ... },
  flush() { ... }
};

```

8. Ingestão de Eventos e Métricas

Recebimento no backend

`SdkService.receiveEvents()` é o fluxo mais crítico do backend. Ele impõe `MAX_EVENTS_PER_REQUEST = 100`, valida obrigatórios, confirma a `siteKey`, confere se site e chave estão ativos, e grava tudo em transação.

```
BEGIN
  upsert browser_sessions(site_key, session_id, visitor_id, user_identifier, context)

  para cada evento aceito:
    validar eventId, type, pageId, url, path, title, occurredAt
    INSERT sdk_events ... ON CONFLICT(event_id) DO NOTHING
    se page_view ou route_change:
      INSERT page_visits ... ON CONFLICT DO NOTHING
    se page_leave:
      UPDATE page_visits com left_at, duration_ms, max_scroll_depth
    se scroll_depth:
      UPDATE page_visits.max_scroll_depth
    se identify:
      UPDATE browser_sessions.user_identifier
COMMIT
```

Consultas agregadas para dashboard

`getStats()` executa várias consultas em paralelo com `Promise.all`. O retorno é um objeto único com seções que o dashboard consome diretamente.

Método	O que mede
<code>getSummary</code>	Eventos totais, page views, visitantes, sessões, ativos, bounce, erros, cliques problemáticos e conversão de forms.
<code>getTopPages</code>	Páginas mais vistas, visitantes e tempo médio.
<code>getTopClicks</code>	Seletores mais clicados.
<code>getProblemInteractions</code>	<code>dead_click</code> e <code>rage_click</code> .
<code>getFormMetrics</code>	Inícios, envios, abandonos, erros de campo e tempo de preenchimento.
<code>getErrors</code>	Erros JS, recurso e API por mensagem e caminho.
<code>getWebVitals</code>	FCP, LCP, CLS, INP quando enviados pelo SDK.
<code>getDevices</code> , <code>getTrafficSources</code>	Dispositivo e origem por contexto.
<code>getNavigationMetrics</code> , <code>getFlowMetrics</code>	Páginas de entrada/saída, rotas comuns e transições.
<code>getFunnelMetrics</code> , <code>getCustomEvents</code>	Funis baseados em metadata e eventos customizados.
<code>getPerformanceMetrics</code> , <code>getContextMetrics</code>	TTFB, load, DOMContentLoaded, navegadores, sistemas, idiomas e timezones.

9. Frontend Angular

O app `frontend` é uma aplicação Angular standalone. A configuração fica em `app.config.ts` e as rotas em `app.routes.ts`.

```
export const appRoutes: Route[] = [
  { path: '', component: LandingComponent, canActivate: [guestGuard] },
  { path: 'auth', component: AuthComponent, canActivate: [guestGuard] },
  { path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] },
  { path: '**', redirectTo: '' },
];
```

Guards

`authGuard` deixa entrar no dashboard se existir `fluxosdk_user_id` no `localStorage`. `guestGuard` faz o inverso: se já está logado, redireciona para `/dashboard`.

Landing

`LandingComponent` monta uma home visual com mock de dashboard em celular. Ele simula eventos vivos por timer, alterna KPIs e direciona para login. É uma peça demonstrativa, não participa diretamente da coleta real.

Autenticação

`AuthComponent` alterna entre login e cadastro com sinais. Ele valida nome, e-mail, senha e confirmação; chama `AuthService`; salva dados do usuário no `localStorage`; e navega para `/dashboard`.

```
localStorage.setItem('fluxosdk_user_id', user.id);
localStorage.setItem('fluxosdk_user_name', user.name || '');
localStorage.setItem('fluxosdk_user_email', user.email);
localStorage.setItem('fluxosdk_user_is_root', String(Boolean(user.is_root)));
```

Serviços HTTP do frontend

Service	Responsabilidade
<code>AuthService</code>	POST register/login.
<code>SitesService</code>	GET/POST/DELETE sites, usando <code>x-user-id</code> .
<code>SdkEventsService</code>	Link do SDK, injeção opcional do script, eventos recentes e estatísticas.
<code>AdminService</code>	Busca overview administrativo root.

Dashboard

`DashboardComponent` é a tela principal. Ele mantém estado com signals, busca sites, seleciona a primeira `siteKey`, busca eventos e métricas, renderiza gráficos Chart.js e atualiza a cada 8 segundos.

```
private async fetchDashboardData() {
  const siteKey = this.selectedSiteKey();
  if (!siteKey) {
```

```

    this.liveEvents.set([]);
    this.stats.set(null);
    return;
  }

  const [events, stats] = await Promise.all([
    this.sdkEventsService.getRecentEvents(siteKey, 40),
    this.sdkEventsService.getStats(siteKey, this.rangeHours()),
  ]);

  this.liveEvents.set(events);
  this.stats.set(stats);
  this.renderCharts();
}

```

Abas do dashboard

- **Dashboard:** KPIs, gráficos, listas analíticas, performance, funis e fluxo.
- **Eventos:** timeline dos eventos recentes.
- **Sessões:** dispositivos, fontes, engajamento e contexto técnico.
- **Sites:** cadastro, listagem, siteKey e exclusão.
- **Admin:** visível só para root, com usuários, sistemas e totais globais.

10. Cliente de Exemplo

`apps/example-client` é o laboratório que valida a integração real. O arquivo `src/index.html` já carrega o SDK usando uma chave pública específica.

```

<script
  src="http://localhost:3000/assets/embed.js"
  data-site-key="pk_ea22c699be734efbb305a4f9316f1cb9"
  data-track-api-errors="true">
</script>

```

Rotas de laboratório

Rota	Eventos gerados
/	Compra de produto, newsletter, <code>identify</code> , eventos customizados de plano e checkout.
/sobre	Link fantasma, clique comum e erro JS manual via <code>trackError</code> .
/contato	Formulário com validação e simulação de erro de API.
/checkout	Funil de checkout com etapas de início, endereço, pagamento, abandono e conversão.
/lab	Cliques repetidos, dead click e evento customizado de onboarding.

Exemplo de evento customizado

```

sdk()?.track('checkout_completed', {
  funnel: 'checkout',
  step: 'checkout_finalizado',

```

```
value: 259.9,  
currency: 'BRL',  
items: 2,  
});
```

Esse app é importante porque exercita quase todas as consultas do dashboard: funis, formulários, cliques, erros, identificação, API error e fluxo de rotas.

11. Fluxos de Ponta a Ponta

Fluxo A: Cadastro e primeiro acesso

```
Usuário -> /auth
  preenche nome, email, senha
  AuthComponent.validateForm()
  AuthService.register()
  POST /api/auth/register
  AuthService backend grava users com password_hash
  frontend salva localStorage
  router.navigate('/dashboard')
```

Fluxo B: Cadastrar site monitorado

```
Dashboard aba Sites
  usuário envia nome + domínio
  SitesService.createSite()
  POST /api/sites com x-user-id
  SitesController valida header
  SitesService limpa domínio e insere sites
  gera publicKey pk_...
  insere site_keys
  dashboard recarrega sites e métricas
```

Fluxo C: Coleta automática de page view

```
example-client carrega index.html
<script src="/assets/embed.js" data-site-key="pk_...">
SDK lê data-site-key
cria visitorId no localStorage
cria sessionId no sessionStorage
cria page snapshot
queueEvent(page_view)
flush -> PUT /api/sdk/events
backend valida chave, grava sessão e evento
se page_view: cria registro em page_visits
```

Fluxo D: Detectar clique problemático

```
Usuário clica
  SDK gera metadata do elemento: selector, tag, text, href, role
  sempre envia click
  se 3 cliques no mesmo selector em 2s -> rage_click
  se elemento não parece acionável -> aguarda 700ms
  se pageId/url não mudaram -> dead_click
  dashboard agrega em getProblemInteractions()
```


Fluxo E: Visualização do dashboard

```
DashboardComponent.ngOnInit()
  extractUserInfo()
  loadSites()
  selectedSiteKey = primeiro site
  fetchDashboardData()
    GET /api/sdk/events/recent
    GET /api/sdk/stats?siteKey=...&rangeHours=24
  stats signal recebe payload
  renderCharts() recria gráficos
  setInterval a cada 8000ms repete busca
```

12. Testes e Qualidade

O repositório tem estrutura de testes criada pelo Nx, mas parte dos testes ainda não foi atualizada para a aplicação real.

Arquivo	Estado observado
apps/frontend/src/app/app.spec.ts	Procura <code>Welcome frontend</code> e importa <code>NxWelcome</code> , enquanto o app real usa <code>router-outlet</code> .
apps/example-client/src/app/app.spec.ts	Procura <code>Welcome example-client</code> , mas o app real exibe laboratório de métricas.
apps/backend-e2e/src/backend/backend.spec.ts	Espera <code>GET /</code> com <code>{ message: 'Hello API' }</code> , rota que não aparece implementada no backend atual.
apps/frontend-e2e/src/example.spec.ts	Espera um <code>h1</code> com <code>Welcome</code> ; a landing atual tem outro conteúdo.

Conclusão de qualidade: a base tem infraestrutura de teste, mas a cobertura não acompanha o produto atual. O próximo passo mais valioso é reescrever testes de backend para `/api/sdk/events`, `/api/sdk/stats`, autenticação e sites, e testes de frontend para auth/dashboard.

13. Pontos de Atenção e Melhorias

Tema	Observação	Sugestão
Autenticação	Uso de <code>x-user-id</code> vindo do cliente.	Adicionar JWT/sessão assinada e middleware de autenticação.
CORS	<code>Access-Control-Allow-Origin: *</code> .	Restringir por domínio cadastrado ou por ambientes.
Configuração	Frontend usa <code>http://localhost:3000</code> hardcoded.	Criar environments Angular ou variáveis por build.
Porta da API	<code>API_PORTA</code> não tem fallback no código.	Usar fallback <code>3000</code> e validar env obrigatória na subida.

Tema	Observação	Sugestão
Texto	Alguns arquivos têm mojibake, por exemplo <code>Usuário</code> .	Normalizar encoding UTF-8 e revisar strings visíveis.
Validação de payload	Valida obrigatórios, mas não há schema formal.	Usar validação estrutural com biblioteca ou camada própria tipada.
Volume	Há limite por request, mas não rate limit.	Adicionar rate limiting por <code>siteKey</code> /IP e compressão.
Privacidade	SDK coleta texto de elementos e URLs.	Adicionar opt-out, máscara de campos sensíveis e política de consentimento.
Testes	Specs ainda refletem scaffold Nx.	Atualizar testes para fluxos reais e adicionar fixtures de banco.
Health check	<code>health.controller.ts</code> está vazio.	Implementar <code>/api/health</code> com status de API e banco.

14. Conclusão

O FluxoSDK já tem uma linha de produto funcional: um script de analytics completo, uma API que persiste eventos e calcula métricas ricas, e um dashboard Angular capaz de contar a história do comportamento do usuário. A espinha dorsal técnica está bem definida.

Os maiores próximos ganhos estão em segurança de autenticação, configuração por ambiente, revisão de encoding/textos, testes reais e controles de privacidade/volume. Com esses ajustes, o projeto deixa de ser apenas um laboratório sólido e se aproxima de uma base pronta para uso controlado.